AKADEMIA NAUK STOSOWANYCH W ELBLĄGU

Instytut Informatyki Stosowanej im. Krzysztofa Brzeskiego

System Obsługi Praktyk Zawodowych

Dokumentacja Techniczna i Funkcjonalna

Spis treści

1. Wstęp i cel projektu *4*
1.1. Cel systemu 4
1.2. Zakres dokumentacji 4
2. Analiza interesariuszy (User Roles) *5*
3. Wymagania funkcjonalne *6*
4. Scenariusze użycia — User Stories *8*
User Story 1 — Prowadzenie Dziennika Praktyki 8
User Story 2 — Weryfikacja i zatwierdzenie dokumentu przez UOPZ 8
User Story 3 — Generowanie Protokołu i ocena końcowa 8
5. Wymagania niefunkcjonalne *10*
5.1. Bezpieczeństwo 10
5.2. Użyteczność 10
5.3. Wydajność 10
5.4. Archiwizacja i trwałość danych 10
6. Architektura systemu *12*
6.1. Stos technologiczny 12
6.2. Struktura projektu 12
6.3. Obsługa danych — faza I (JSON) 13
7. Role i uprawnienia użytkowników *14*
7.1. Definicje ról 14
7.2. Implementacja kontroli dostępu 14
7.3. Macierz uprawnień do załączników 14
8. Przepływ procesu praktyki zawodowej *16*
8.1. Ścieżka standardowa 16
8.2. Ścieżka alternatywna — uznanie pracy zawodowej / stażu 16
8.3. Cykl życia dokumentu Dziennik Praktyki (Zał. 6) 17
9. Statusy praktyki w systemie *18*
10. Opis modułów i formularzy *19*
10.1. Moduł autoryzacji — blueprint: auth 19
10.2. Moduł studenta — blueprint: student 19
10.3. Formularz Zał. 6 — Dziennik praktyki (dynamiczna tabela) 19
10.4. Formularz Zał. 4 — Potwierdzenie efektów uczenia się 19
10.5. Moduł UOPZ — blueprint: uopz 20
10.6. Moduł ZOPZ — blueprint: zopz 20

10.7. Moduł dziekanatu — blueprint: dziekanat 20

11. Status projektu i plan prac 21

12. Projekt bazy danych 22

12.1. Wybór systemu bazy danych 22

Faza I — SQLite (prototyp) 22

Faza II — PostgreSQL (produkcja) 22

12.2. Narzędzie do prototypowania bazy danych — DBeaver Community 23

12.3. Narzędzie do modelowania i dokumentowania — Mermaid 23

12.4. Schemat bazy danych — opis tabel 24

1. Wstęp i cel projektu

Niniejszy dokument stanowi dokumentację techniczną i funkcjonalną systemu obsługi praktyk zawodowych dla kierunku Informatyka w Instytucie Informatyki Stosowanej im. Krzysztofa Brzeskiego ANS w Elblągu.

Obecny proces obsługi praktyk opiera się na 11 papierowych załącznikach, które muszą być ręcznie drukowane, wypełniane, podpisywane i dostarczane do odpowiednich osób. Generuje to szereg problemów: zagubione dokumenty, brak wglądu w status praktyki, konieczność fizycznej obecności w dziekanacie, brak historii zmian.

1.1. Cel systemu

System ma na celu pełną digitalizację i automatyzację obiegu dokumentów związanych z praktykami zawodowymi. Kluczowe założenia:

- Zastąpienie papierowych załączników cyfrowymi formularzami wypełnianymi przez przeglądarkę
- Kontrola dostępu oparta na rolach — każdy użytkownik widzi tylko to, do czego ma uprawnienia
- Śledzenie statusu praktyki w czasie rzeczywistym
- Możliwość generowania gotowych dokumentów PDF na podstawie wprowadzonych danych
- Powiadamianie uczestników procesu o wymaganych działaniach przez e-mail

1.2. Zakres dokumentacji

Dokument obejmuje: analizę interesariuszy, wymagania funkcjonalne i niefunkcjonalne, User Stories, opis architektury systemu, specyfikację ról i uprawnień, opis przepływu procesu, opis wszystkich obsługiwanych formularzy oraz szczegóły implementacyjne dla poszczególnych modułów aplikacji Flask.

2. Analiza interesariuszy (User Roles)

Na podstawie analizy regulaminu praktyk zawodowych oraz procesów biznesowych zidentyfikowano czterech kluczowych aktorów systemu. Każdy aktor posiada odmienne cele i potrzeby, które system musi spełnić.

Aktor	Identyfikator	Główny cel w systemie	Kluczowe potrzeby
Student	student	Sprawne wypełnienie i złożenie dokumentacji praktyki zawodowej	Podgląd statusu praktyki, wypełnianie formularzy online, brak konieczności drukowania dokumentów
Uczelniany Opiekun Praktyk (UOPZ)	uopz	Nadzór nad przebiegiem praktyk, ocena studentów i prowadzenie egzaminu	Przegląd dokumentacji wszystkich studentów, możliwość dodawania uwag, wystawianie ocen i protokołów
Zakładowy Opiekun Praktyk (ZOPZ)	zopz	Potwierdzenie wykonanych zadań i ocena studenta w miejscu praktyki	Prosty dostęp do dziennika studenta, możliwość elektronicznego potwierdzania wpisów bez logowania do systemu uczelnianego
Sekretariat / Dziekanat / Dyrektor	dziekanat	Formalna obsługa dokumentacji: porozumienia, oświadczenia, decyzje administracyjne	Centralne repozytorium dokumentów, podpisywanie porozumień, zarządzanie wnioskami o uznanie efektów uczenia się

3. Wymagania funkcjonalne

Wymagania funkcjonalne opisują, co system ma robić. Każde wymaganie sformułowane jest w postaci: System umożliwia [aktorowi] na [czynność]. Priorytety nadano zgodnie z metodą MoSCoW: Must Have (konieczne), Should Have (powinno być), Could Have (może być), Won't Have (poza zakresem).

ID	Nazwa	Opis wymagania	Aktor	MoSCoW
WF-01	Logowanie i autoryzacja	System umożliwia każdemu użytkownikowi zalogowanie się na swoje konto i uzyskanie dostępu zgodnego z przypisaną rolą.	Wszyscy	Must Have
WF-02	Prowadzenie dziennika praktyki	System umożliwia studentowi codzienne wypełnianie Dziennika Praktyki (Zał. 6) w formie dynamicznej tabeli z możliwością dodawania i usuwania wierszy bez przeładowania strony.	Student	Must Have
WF-03	Blokowanie edycji po przesłaniu	System umożliwia automatyczne zablokowanie edycji dokumentu po jego przesłaniu do weryfikacji — student może edytować wyłącznie dokumenty o statusie Draft lub Rejected.	System	Must Have
WF-04	Potwierdzanie wpisów przez ZOPZ	System umożliwia Zakładowemu Opiekunowi Praktyk potwierdzanie poszczególnych wpisów w Dzienniku Praktyki elektronicznym podpisem.	ZOPZ	Must Have
WF-05	Generowanie dokumentów PDF	System umożliwia wygenerowanie i pobranie każdego załącznika w formacie PDF na podstawie danych wprowadzonych przez użytkownika.	Wszyscy	Must Have
WF-06	Potwierdzenie efektów uczenia się	System umożliwia Zakładowemu Opiekunowi Praktyk wypełnienie formularza potwierdzenia uzyskania efektów uczenia się (Zał. 4) z przypisaniem wyniku do każdego efektu.	ZOPZ	Must Have
WF-07	Wystawianie oceny końcowej i protokołu	System umożliwia Uczelnianemu Opiekunowi Praktyk wystawienie oceny końcowej i wygenerowanie Protokołu zaliczenia praktyki (Zał. 8) po przeprowadzeniu egzaminu ustnego.	UOPZ	Must Have
WF-08	Śledzenie statusu praktyki	System umożliwia studentowi i opiekunowi bieżący podgląd aktualnego statusu praktyki oraz listy dokumentów wymagających uzupełnienia.	Student, UOPZ	Must Have
WF-09	Dodawanie uwag przez opiekunów	System umożliwia Uczelnianemu Opiekunowi Praktyk dodawanie uwag tekstowych do poszczególnych sekcji dokumentów w przypadku ich odrzucenia.	UOPZ	Should Have
WF-10	Powiadomienia e-mail	System umożliwia automatyczne wysyłanie powiadomień e-mail do	System	Should Have

ID	Nazwa	Opis wymagania	Aktor	MoSCoW
		zainteresowanych stron przy kluczowych zmianach statusu dokumentu.		
WF-11	Wniosek o zaliczenie pracy zawodowej	System umożliwia studentowi złożenie wniosku o zaliczenie praktyki na podstawie pracy zawodowej, stażu lub działalności gospodarczej (Zał. 4b).	Student	Should Have
WF-12	Zarządzanie użytkownikami	System umożliwia administratorowi tworzenie, edycję i dezaktywację kont użytkowników oraz przypisywanie ról.	Admin	Should Have
WF-13	Historia zmian dokumentu	System umożliwia przeglądanie historii zmian każdego dokumentu z datą, autorem zmiany i poprzednim statusem.	UOPZ, Admin	Could Have
WF-14	Eksport danych do CSV	System umożliwia administratorowi i dziekanatowi eksport zbiorczych danych o praktykach do pliku CSV.	Dziekanat, Admin	Could Have

4. Scenariusze użycia — User Stories

Poniżej przedstawiono trzy kluczowe funkcjonalności systemu opisane w formacie User Story: Jako [rola], chcę [cel], aby [korzyść].

User Story 1 — Prowadzenie Dziennika Praktyki

Jako student, chcę codziennie wypełniać Dziennik Praktyki online, aby nie musieć drukować i ręcznie uzupełniać papierowego dziennika oraz mieć pewność, że moje wpisy są bezpiecznie zapisane.

Kryteria akceptacji:

- Student moze dodawac nowy wiersz do tabeli dziennika bez przeladowania strony (przycisk Dodaj dzien)
- Każdy wiersz zawiera pola: data, opis wykonanych prac, numer efektu uczenia się
- Student może usunąć błędnie dodany wiersz przed wysłaniem
- Po klikniecіu Wyslij do weryfikacji formularz zostaje zablokowany do edycji
- ZOPZ otrzymuje powiadomienie e-mail o nowym wpisie wymagającym potwierdzenia

User Story 2 — Weryfikacja i zatwierdzenie dokumentu przez UOPZ

Jako Uczelniany Opiekun Praktyk, chcę przeglądać przesłane dokumenty studentów i dodawać uwagi do odrzucanych sekcji, aby student wiedział dokładnie, co wymaga poprawy, bez konieczności osobistego spotkania.

Kryteria akceptacji:

- UOPZ widzi listę wszystkich dokumentów oczekujących na weryfikację z datą przesłania
- UOPZ może dodać uwagę tekstową do każdej sekcji dokumentu przed odrzuceniem
- Po odrzuceniu student otrzymuje powiadomienie e-mail z treścią uwag
- Student może ponownie edytować dokument o statusie Rejected i wysłać poprawioną wersję
- Po zatwierdzeniu status dokumentu zmienia się na Approved i dalsze zmiany są zablokowane

User Story 3 — Generowanie Protokołu i ocena końcowa

Jako Uczelniany Opiekun Praktyk, chcę po egzaminie ustnym wprowadzić oceny składowe i wygenerować gotowy Protokół zaliczenia praktyki (Zał. 8) w formacie PDF, aby nie wypełniać dokumentu ręcznie i od razu dołączyć go do akt studenta.

Kryteria akceptacji:

- UOPZ może wprowadzić trzy oceny składowe: S (za sprawozdanie), U (ocena UOPZ), Z (ocena ZOPZ)
- System automatycznie wylicza ocenę końcową według ustalonego algorytmu

- Po zatwierdzeniu system generuje plik PDF Protokołu z wypełnionymi danymi studenta, zakładu i ocenami
- Protokół trafia do cyfrowych akt studenta i jest dostępny dla dziekanatu
- UOPZ może pobrać PDF i wydrukować go do akt papierowych

5. Wymagania niefunkcjonalne

Wymagania niefunkcjonalne (jakościowe) określają, jak system ma działać, a nie co ma robić. Podzielono je na cztery kategorie.

5.1. Bezpieczeństwo

- Wszystkie hasła przechowywane jako skróty kryptograficzne (bcrypt przez Flask-Bcrypt)
- Każdy formularz chroniony tokenem CSRF (Flask-WTF) — ochrona przed atakami Cross-Site Request Forgery
- Wszystkie trasy wymagające logowania chronione dekoratorem @login_required
- Dostęp do danych innych użytkowników blokowany przez @role_required — student nie może zobaczyć dokumentów innego studenta
- Pliki JSON i PDF niedostępne bezpośrednio przez URL — serwowane wyłącznie przez Flask po weryfikacji uprawnień
- Sesja użytkownika wygasa po 30 minutach bezczynności

5.2. Użyteczność

- Interfejs responsywny — poprawne działanie na komputerach, tabletach i urządzeniach mobilnych (Bootstrap 5)
- Dynamiczne dodawanie wierszy w tabelach formularzy bez przeładowania strony (JavaScript)
- Czytelny panel statusu praktyki dla każdej roli — użytkownik widzi co musi zrobić jako następny krok
- Powiadomienia e-mail przy kluczowych zmianach statusu dokumentu
- Komunikaty błędów walidacji wyświetlane bezpośrednio przy polach formularza

5.3. Wydajność

- Czas generowania dokumentu PDF: poniżej 3 sekund dla standardowego formularza
- Czas odpowiedzi serwera na żądanie HTTP: poniżej 500 ms dla operacji odczytu danych
- System powinien obsługiwać jednoczesną pracę co najmniej 50 użytkowników bez zauważalnego spadku wydajności

5.4. Archiwizacja i trwałość danych

- Faza I: dane przechowywane w plikach JSON — każdy dokument w osobnym pliku powiązanym z numerem albumu studenta
- Faza II (planowana): migracja do relacyjnej bazy danych (PostgreSQL / SQLite) zapewniającej integralność danych
- Pliki JSON tworzone z kodowaniem UTF-8 i wcięciami (indent=4) dla czytelności i możliwości ręcznej edycji

- Wygenerowane pliki PDF archiwizowane na serwerze w katalogu powiązanym z ID praktyki studenta
- Dane nie są usuwane — dokumenty odrzucone i poprzednie wersje pozostają dostępne dla administratora

6. Architektura systemu

6.1. Stos technologiczny

System zbudowany jest w architekturze klient-serwer. Backend oparty jest na frameworku Flask (Python), komunikacja z przeglądarką odbywa się przez HTTP, a widoki renderowane są po stronie serwera przy użyciu szablonów Jinja2.

Warstwa	Technologia	Opis zastosowania
Backend	Python 3.11+, Flask	Główna logika aplikacji, routing, przetwarzanie formularzy
Autoryzacja	Flask-Login	Zarządzanie sesją, ochrona tras, tożsamość użytkownika
Formularze	Flask-WTF, WTForms	Walidacja danych, ochrona CSRF, dynamiczne pola listowe
Szablony	Jinja2	Renderowanie HTML po stronie serwera, pętle i warunki
Frontend	Bootstrap 5, JavaScript	Responsywny interfejs, dynamiczne wiersze tabel (addRow/deleteRow)
Baza danych (faza I)	JSON (load_data / save_data)	Zapis formularzy do plików .json per student
Baza danych (faza II)	PostgreSQL / SQLite	Docelowe środowisko produkcyjne (planowane)
Generowanie PDF	WeasyPrint / ReportLab	Eksport wypełnionych formularzy do PDF
Powiadomienia	Flask-Mail	E-mail do opiekunów, studentów, dziekanatu

6.2. Struktura projektu

Projekt podzielony jest na blueprinty Flask — każda rola posiada własny moduł z trasami i formularzami. Dane przechowywane są w plikach JSON z podziałem na typ załącznika i numer albumu studenta.

Struktura katalogów

```
praktyki-system/
├── app/
│   ├── __init__.py              # Fabryka aplikacji Flask
│   ├── config.py                # Konfiguracja środowisk (dev/prod)
│   ├── blueprints/
│   │   ├── auth/                # Logowanie, wylogowanie, reset hasła
│   │   ├── student/             # Widoki i formularze studenta
│   │   ├── uopz/                # Uczelniany opiekun praktyk
│   │   ├── zopz/                # Zakładowy opiekun praktyk
│   │   ├── dziekanat/           # Dziekanat / Dyrektor Instytutu
│   │   ├── admin/               # Zarządzanie użytkownikami i rolami
│   │   └── dokumenty/           # Generowanie i pobieranie PDF
│   ├── forms/                   # Formularze Flask-WTF / WTForms
```

```
│   ├── templates/              # Szablony Jinja2
│   └── static/                 # CSS, JS, Bootstrap
├── data/                       # Pliki JSON (faza I)
│   ├── users.json
│   ├── praktyki.json
│   ├── zal4_efekty/            # Zał. 4 per student
│   └── zal6_dziennik/          # Zał. 6 per student
├── dokumentacja/
├── requirements.txt
└── run.py
```

6.3. Obsługa danych — faza I (JSON)

W pierwszej fazie projektu dane przechowywane są w plikach JSON. Każdy formularz zapisywany jest do osobnego pliku powiązanego z numerem albumu studenta.

Funkcje pomocnicze — app/utils.py

```
import json, os

def load_data(path):
    if os.path.exists(path):
        with open(path, 'r', encoding='utf-8') as f:
            return json.load(f)
    return []

def save_data(path, data):
    os.makedirs(os.path.dirname(path), exist_ok=True)
    with open(path, 'w', encoding='utf-8') as f:
        json.dump(data, f, indent=4, ensure_ascii=False)
```

7. Role i uprawnienia użytkowników

System obsługuje pięć ról. Dostęp do dokumentu jest przyznawany na poziomie widoku (podgląd) lub edycji. Użytkownik wymieniony w dokumencie zawsze może go zobaczyć, ale nie zawsze edytować.

7.1. Definicje ról

Rola	Identyfikator	Opis
Student	student	Osoba odbywająca praktykę. Wypełnia wnioski, prowadzi dziennik, pisze sprawozdanie i ankietę.
Uczelniany Opiekun Praktyk	uopz	Pracownik uczelni. Wydaje skierowania, nadzoruje przebieg, przeprowadza egzamin, wpisuje ocenę do USOS.
Zakładowy Opiekun Praktyk	zopz	Pracownik zakładu pracy. Potwierdza wpisy w dzienniku, podpisuje kartę praktyki i sprawozdanie.
Dziekanat / Dyrektor	dziekanat	Podpisuje porozumienia, przyjmuje oświadczenia, podejmuje decyzje o uznaniu efektów uczenia się.
Administrator	admin	Pełny dostęp do systemu. Zarządza użytkownikami, rolami i konfiguracją.

7.2. Implementacja kontroli dostępu

Kontrola dostępu realizowana jest przez dekoratory Flask-Login oraz własny dekorator @role_required. Każda trasa jest chroniona — nieautoryzowany dostęp zwraca błąd 403.

Dekorator @role_required — app/auth/decorators.py

```
from functools import wraps
from flask import abort
from flask_login import current_user

def role_required(*roles):
    def decorator(f):
        @wraps(f)
        def decorated(*args, **kwargs):
            if not current_user.is_authenticated:
                abort(401)
            if current_user.rola not in roles:
                abort(403)
            return f(*args, **kwargs)
        return decorated
    return decorator
```

7.3. Macierz uprawnień do załączników

Załącznik	Student	UOPZ	ZOPZ	Dziekanat	Admin
Zał. 1 – Porozumienie z zakładem pracy	V	V	—	E / Podpis	E
Zał. 2 – Program praktyki	V	E	V	V	E
Zał. 2a – Program i harmonogram	V	E	V	V	E
Zał. 3 – Karta praktyki + Skierowanie	V	E (skierowanie)	E (opinia)	V	E
Zał. 4 – Potwierdzenie efektów uczenia się	V	V	E	V	E
Zał. 4a – Ocena wniosku studenta	—	E	—	E (decyzja)	E
Zał. 4b – Wniosek studenta	E	V	—	V	E
Zał. 5 – Kwestionariusz ankiety	E	V	—	—	V
Zał. 6 – Dziennik praktyki	E (wpisy)	V	E (podpis)	V	E
Zał. 7 / 7a – Sprawozdanie	E	V	E (podpis)	V	E
Zał. 8 – Protokół egzaminu	V (własny)	E	—	E	E
Zał. 9 – Oświadczenie instytucji	V	V	—	E	E

8. Przepływ procesu praktyki zawodowej

8.1. Ścieżka standardowa

Krok	Działanie	Rola	Załącznik	Status w systemie
1	Student wyszukuje zakład pracy i uzyskuje potwierdzenie przyjęcia	Student	Zał. 9	OCZEKUJE_NA_ZAL9
2	Student dostarcza Zał. 9 do dziekanatu	Dziekanat	Zał. 9	ZAL9_PRZYJETY
3	Uczelnia podpisuje porozumienie z zakładem i wysyła Regulamin + Zał. 2	Dziekanat	Zał. 1, 2	POROZUMIENIE_PODPISANE
4	UOPZ, ZOPZ i student uzgadniają program i harmonogram	UOPZ, ZOPZ, Student	Zał. 2a	PROGRAM_UZGODNIONY
5	UOPZ wydaje skierowanie na praktykę	UOPZ	Zał. 3	SKIEROWANIE_WYDANE
6	Student rozpoczyna praktykę (960h / 120 dni / 6 miesięcy)	Student	—	PRAKTYKA_W_TOKU
7	Student codziennie wypełnia dziennik; ZOPZ potwierdza wpisy	Student, ZOPZ	Zał. 6	PRAKTYKA_W_TOKU
8	UOPZ przeprowadza minimum 1 hospitację w zakładzie	UOPZ	—	PRAKTYKA_W_TOKU
9	Do 7 dni po zakończeniu student składa komplet dokumentów	Student	Zał. 3, 4, 6, 7, 5	DOKUMENTY_ZLOZONE
10	Egzamin ustny przed komisją egzaminacyjną	UOPZ + Komisja	Zał. 8	EGZAMIN
11	Komisja sporządza protokół zaliczenia	UOPZ + Komisja	Zał. 8	PROTOKOL_SPORZADZONY
12	UOPZ wpisuje ocenę do systemu USOS	UOPZ	—	ZALICZONA

8.2. Ścieżka alternatywna — uznanie pracy zawodowej / stażu

1. Student składa wniosek — Zał. 4b (Wniosek o zaliczenie efektów uczenia się)
2. Do wniosku dołącza: zaświadczenie o zatrudnieniu, opis stanowiska pracy, zaświadczenie z CEIDG lub odpis KRS
3. Komisja ds. praktyk dokonuje merytorycznej oceny wniosku — Zał. 4a
4. Ostateczną decyzję podejmuje Dyrektor Instytutu
5. Decyzja pozytywna: UOPZ wpisuje zaliczenie do USOS
6. Decyzja negatywna: student realizuje praktykę ścieżką standardową

8.3. Cykl życia dokumentu Dziennik Praktyki (Zał. 6)

Poniżej opisano pełny cykl życia dokumentu Dziennik Praktyki — od momentu utworzenia przez studenta, przez weryfikację i możliwe poprawki, aż do ostatecznego zatwierdzenia przez Uczelnianego Opiekuna Praktyk.

Etap	Status dokumentu	Aktor	Opis działania
1. Utworzenie	Draft	Student	Student tworzy nowy dziennik w systemie. Formularz jest edytowalny — student może dodawać, edytować i usuwać wiersze tabeli.
2. Uzupełnianie	Draft	Student + ZOPZ	Student codziennie dodaje wpis z opisem prac i numerem efektu uczenia się. ZOPZ potwierdza każdy wpis elektronicznie.
3. Przeslanie do weryfikacji	Submitted	Student	Student klika przycisk Wyslij. Formularz zostaje zablokowany do edycji. System wysyla powiadomienie e-mail do UOPZ.
4. Przegląd przez UOPZ	Under_Review	UOPZ	UOPZ przegląda dziennik. Może zatwierdzić lub odrzucić dokument z uwagami tekstowymi do konkretnych sekcji.
5a. Odrzucenie z uwagami	Rejected	UOPZ	UOPZ odrzuca dokument i dodaje uwagi. System wysyła powiadomienie e-mail do studenta z treścią uwag. Formularz zostaje odblokowany.
5b. Zatwierdzenie	Approved	UOPZ	UOPZ zatwierdza dziennik. Status zmienia się na Approved. Dokument jest zablokowany — dalsze zmiany niemożliwe.
6. Poprawki studenta	Draft	Student	Student wprowadza poprawki zgodnie z uwagami UOPZ i ponownie przesyła dokument do weryfikacji (powrót do etapu 3).
7. Zamknięcie	Closed	System	Po zaliczeniu praktyki i wpisaniu oceny do USOS wszystkie dokumenty studenta zmieniają status na Closed i trafiają do archiwum.

9. Statusy praktyki w systemie

Status	Opis	Przejście do następnego
OCZEKUJE_NA_ZAL9	Student zarejestrowany, brak oświadczenia instytucji	Student dostarcza Zał. 9
ZAL9_PRZYJETY	Dziekanat przyjął Zał. 9	Dziekanat podpisuje Zał. 1
POROZUMIENIE_PODPISANE	Porozumienie z zakładem podpisane	Uzgodnienie programu (Zał. 2a)
PROGRAM_UZGODNIONY	Program i harmonogram zatwierdzony	UOPZ wydaje skierowanie
SKIEROWANIE_WYDANE	Skierowanie wystawione przez UOPZ	Student rozpoczyna praktykę
PRAKTYKA_W_TOKU	Praktyka trwa — student prowadzi dziennik	Student składa dokumenty po zakończeniu
DOKUMENTY_ZLOZONE	Komplet dokumentów złożony do UOPZ	Komisja przeprowadza egzamin
EGZAMIN	Egzamin ustny przed komisją w toku	Komisja sporządza protokół
PROTOKOL_SPORZADZONY	Protokół zaliczenia podpisany	UOPZ wpisuje ocenę do USOS
ZALICZONA	Praktyka zaliczona, ocena wpisana do USOS	— (stan końcowy)
ODRZUCONA	Wniosek o zaliczenie pracy odrzucony przez Dyrektora	Student wraca do ścieżki standardowej
PRZEDLUZONA	Praktyka przedłużona na wniosek studenta (max 1 miesiąc)	Powrót do PRAKTYKA_W_TOKU

10. Opis modułów i formularzy

10.1. Moduł autoryzacji — blueprint: auth

Trasa	Metoda	Opis
/login	GET, POST	Formularz logowania — email + hasło. Po poprawnym logowaniu przekierowanie według roli.
/logout	GET	Wylogowanie użytkownika, czyszczenie sesji.
/reset-hasla	GET, POST	Formularz do resetowania hasła przez e-mail.

10.2. Moduł studenta — blueprint: student

Trasa	Metoda	Opis
/student/dashboard	GET	Strona główna — status praktyki, lista dokumentów do uzupełnienia.
/student/zal4b	GET, POST	Formularz wniosku o zaliczenie pracy zawodowej (Zał. 4b).
/student/zal5	GET, POST	Kwestionariusz ankiety oceniający przebieg praktyki (Zał. 5).
/student/zal6	GET, POST	Dziennik praktyki — dynamiczna tabela wierszy z opisem prac (Zał. 6).
/student/zal7	GET, POST	Sprawozdanie z praktyki zawodowej (Zał. 7 lub 7a).
/student/dokumenty	GET	Lista wszystkich dokumentów studenta z ich statusami.

10.3. Formularz Zał. 6 — Dziennik praktyki (dynamiczna tabela)

Dziennik praktyki ma formę tabeli, w której każdy wiersz odpowiada jednemu dniu pracy. Student może dodawać wiersze dynamicznie bez przeładowania strony. ZOPZ potwierdza każdy wpis cyfrowym podpisem.

Przykład struktury JSON — Zał. 6

```
{  "nr_albumu": "12345",  "miejsce_praktyki": "Nazwa Firmy Sp. z o.o.",
   "data_rozpoczecia": "2024-10-01",  "data_zakonczenia": "2025-02-14",
   "wpisy": [{ "dzien": 1, "data": "2024-10-01",
     "opis": "Zapoznanie z infrastrukturą sieci...",
     "nr_efektu": "EK_01, EK_03", "potwierdzenie_zopz": false }] }
```

10.4. Formularz Zał. 4 — Potwierdzenie efektów uczenia się

Formularz wypełniany przez ZOPZ po zakończeniu praktyki. Zawiera listę efektów uczenia się — ZOPZ zaznacza przy każdym efekcie, czy student go osiągnął.

10.5. Moduł UOPZ — blueprint: uopz

Trasa	Metoda	Opis
/uopz/studenci	GET	Lista wszystkich studentów z aktualnym statusem praktyki.
/uopz/skierowanie/<id>	GET, POST	Wydanie skierowania na praktykę (Zał. 3).
/uopz/program/<id>	GET, POST	Edycja programu i harmonogramu praktyki (Zał. 2a).
/uopz/hospitacja/<id>	GET, POST	Rejestracja hospitacji w zakładzie pracy.
/uopz/egzamin/<id>	GET, POST	Wystawienie oceny i sporządzenie protokołu (Zał. 8).
/uopz/efekty/<id>	GET	Podgląd i uznanie efektów uczenia się (Zał. 4).

10.6. Moduł ZOPZ — blueprint: zopz

Trasa	Metoda	Opis
/zopz/dziennik/<id>	GET, POST	Podgląd dziennika studenta — potwierdzanie wpisów (Zał. 6).
/zopz/efekty/<id>	GET, POST	Wypełnienie potwierdzenia efektów uczenia się (Zał. 4).
/zopz/karta/<id>	GET, POST	Podpisanie karty praktyki i wystawienie opinii (Zał. 3).
/zopz/sprawozdanie/<id>	GET, POST	Podpisanie sprawozdania studenta (Zał. 7 / 7a).

10.7. Moduł dziekanatu — blueprint: dziekanat

Trasa	Metoda	Opis
/dziekanat/porozumienia	GET	Lista porozumień z zakładami pracy — podgląd i podpisanie (Zał. 1).
/dziekanat/oswiadczenia	GET, POST	Przyjęcie oświadczenia instytucji dostarczonego przez studenta (Zał. 9).
/dziekanat/wnioski	GET	Lista wniosków o zaliczenie pracy zawodowej (Zał. 4b + 4a).
/dziekanat/komisja	GET, POST	Zarządzanie składem komisji egzaminacyjnej.

11. Status projektu i plan prac

Funkcjonalność	Status	Priorytet
Diagramy architektoniczne i dokumentacja	Gotowe	Wysoki
Struktura projektu Flask i konfiguracja	W trakcie	Wysoki
System logowania i zarządzanie rolami	Planowane	Wysoki
Formularz Zał. 4 — Potwierdzenie efektów	Planowane	Wysoki
Formularz Zał. 6 — Dziennik (dyn. tabela)	Planowane	Wysoki
Pozostałe załączniki (1, 2, 3, 5, 7, 8, 9)	Planowane	Średni
Generowanie PDF z formularzy	Planowane	Średni
Powiadomienia e-mail	Planowane	Niski
Migracja z JSON do bazy danych (faza II)	Planowane	Niski

12. Projekt bazy danych

Rozdział opisuje decyzje projektowe dotyczące warstwy danych systemu: wybór silnika bazy danych, narzędzia do prototypowania i modelowania oraz schemat ERD wraz z uzasadnieniem struktury tabel.

12.1. Wybór systemu bazy danych

Faza I — SQLite (prototyp)

Na początkowym etapie projektu zastosowano pliki JSON jako warstwę danych (load_data / save_data). Naturalnym kolejnym krokiem przed pełną bazą relacyjną jest SQLite — standardowa biblioteka Pythona, która nie wymaga żadnej konfiguracji ani osobnego serwera. Cała baza to jeden plik app.db, który można commitować razem z kodem do repozytorium.

Zalety SQLite w fazie prototypu:

- Wbudowany w Pythona — zero dodatkowych zależności
- Nie wymaga instalacji serwera bazodanowego
- Jeden plik .db — prosty backup i przenoszenie projektu
- Pełna integracja z Flask-SQLAlchemy bez zmiany kodu aplikacji
- Wystarczający dla środowiska deweloperskiego i testów

Ograniczenia SQLite — dlaczego nie na produkcji:

- Brak obsługi wielu jednoczesnych operacji zapisu (jeden writer na raz)
- Ograniczone wsparcie ALTER TABLE — trudniejsze migracje schematu
- Brak wbudowanego systemu uprawnień na poziomie bazy
- Nie nadaje się do środowiska z wieloma równoczesnymi użytkownikami

Faza II — PostgreSQL (produkcja)

Jako docelowy silnik bazy danych dla wersji produkcyjnej wybrano PostgreSQL. Decyzja zapadła po porównaniu z MariaDB — PostgreSQL jest lepszym wyborem w ekosystemie Pythona i Flask z kilku kluczowych powodów.

Dlaczego PostgreSQL, a nie MariaDB:

- Natywne wsparcie Flask-SQLAlchemy i popularnych bibliotek Pythona (psycopg2, asyncpg)
- Zaawansowane typy danych: JSONB (natywny JSON z indeksowaniem), ARRAY, UUID
- Pełna zgodność ze standardem SQL — mniej niespodzianek przy migracji z SQLite
- Lepsze narzędzia do migracji: Flask-Migrate (Alembic) jest projektowane przede wszystkim pod PostgreSQL
- Darmowy i open source bez ograniczeń licencyjnych nawet na produkcji
- Szeroko stosowany w projektach Django i Flask — duza baza wiedzy i tutoriali
- Obsługiwany przez wszystkie popularne platformy PaaS: Heroku, Railway, Render, Supabase

Migracja z SQLite do PostgreSQL sprowadza się do zmiany jednej linii w config.py i uruchomienia flask db upgrade — reszta kodu aplikacji pozostaje bez zmian dzięki warstwie abstrakcji SQLAlchemy.

Cecha	SQLite (faza I)	PostgreSQL (faza II)
Instalacja	Zero — wbudowany w Python	Wymaga serwera (lokalnie lub PaaS)
Konfiguracja	Jeden plik app.db	Connection string w config.py
Rownoczesne zapisy	Ograniczone (jeden writer)	Pelne wsparcie wielu polaczen
Migracje schematu	Ograniczony ALTER TABLE	Pelny ALTER TABLE, Flask-Migrate
Typy danych	TEXT, INTEGER, REAL, BLOB	Bogaty zestaw + JSONB, ARRAY, UUID
Srodowisko docelowe	Lokalnie / prototyp / testy	Produkcja, staging, CI/CD
Integracja z Flask	Flask-SQLAlchemy (sqlite:///)	Flask-SQLAlchemy (postgresql://)
Koszt	Darmowy	Darmowy (open source)
Zmiana w kodzie	connection string	— (kod aplikacji bez zmian)

12.2. Narzędzie do prototypowania bazy danych — DBeaver Community

DBeaver Community to darmowe, wieloplatformowe narzędzie graficzne do zarządzania bazami danych. Zostało wybrane jako alternatywa dla MySQL Workbench, ponieważ obsługuje jednocześnie SQLite (faza I) i PostgreSQL (faza II) — nie trzeba zmieniać środowiska pracy przy migracji między fazami projektu.

Zastosowanie DBeaver w projekcie:

- Podgląd struktury tabel po uruchomieniu Flask i wykonaniu flask db upgrade
- Edytor SQL z podpowiadaniem skladni — testowanie zapytan bez uruchamiania aplikacji
- Wizualny podglad relacji miedzy tabelami (widok diagramu ER)
- Import i eksport danych testowych w formacie CSV lub SQL
- Porownanie schematu miedzy baza deweloperska a produkcyjna
- Praca z SQLite lokalnie i pozniej z PostgreSQL na serwerze — bez zmiany narzedzia

Instalacja: DBeaver Community dostepny bezplatnie na dbeaver.io/download dla systemow Windows, macOS i Linux. Polaczenie z SQLite: File > New Database Connection > SQLite > wskazanie pliku app.db.

12.3. Narzędzie do modelowania i dokumentowania — Mermaid

Mermaid to narzedzie tekstowe do tworzenia diagramow, uzywane juz w tym projekcie do diagramow architektury, przepływu procesu i uprawnien. Na potrzeby projektu bazy danych Mermaid sluzy rowniez do tworzenia diagramow ERD (erDiagram).

Dlaczego Mermaid do organizacji projektu bazy danych:

- Kod diagramu ERD przechowywany razem z kodem projektu w repozytorium GitHub
- GitHub renderuje diagramy Mermaid automatycznie w plikach .md — zero dodatkowego oprogramowania
- Jeden format obsluguje ERD, flowcharty, diagramy sekwencji i stanow — wszystkie diagramy projektu w jednym miejscu
- Latwa wspolpraca — diagram to zwykly tekst, mozna go roznicowac (diff) i przeglądac w pull requestach
- Edytor online mermaid.live do podgladu na zywo bez instalacji

Diagram ERD systemu (kod Mermaid) oraz pozostale diagramy projektu przechowywane sa w folderze dokumentacja/diagramy/ w repozytorium GitHub.

12.4. Schemat bazy danych — opis tabel

Ponizej przedstawiono zestawienie wszystkich tabel bazy danych wraz z ich przeznaczeniem i kluczowymi kolumnami. Pelny skrypt SQL (CREATE TABLE) zawarty jest w pliku praktyki_schema.sql w repozytorium.

Tabela	Przeznaczenie	Klucz glowny	Kluczowe relacje
uzytkownik	Konta uzytkownikow systemu wszystkich rol	id	— (tabela bazowa)
student	Profil studenta z numerem albumu i danymi kierunku	id	uzytkownik_id → uzytkownik
zaklad_pracy	Dane zakładu pracy przyjmujacego studenta na praktyke	id	zopz_id → uzytkownik
praktyka	Glowna encja laczaca studenta, zaklad i opiekuna. Trzyma status.	id	student_id, zaklad_id, uopz_id
dokument	Ogolna tabela dla wszystkich 13 zalacznikow (ZAL1–ZAL9)	id	praktyka_id → praktyka
wpis_dziennika	Wiersze tabeli Dziennika Praktyki (Zal. 6) — jeden wiersz = jeden dzien	id	dokument_id → dokument
efekt_uczenia	Efekty uczenia sie z Zal. 4 — jeden wiersz = jeden efekt EK_XX	id	dokument_id → dokument
protokol	Oceny egzaminu i protokol zaliczenia praktyki (Zal. 8)	id	praktyka_id → praktyka (1:1)
porozumienie	Porozumienie z zakladem pracy (Zal. 1) — jeden na praktyke	id	praktyka_id, zaklad_id
historia_statusu	Pelna historia zmian statusu praktyki z autorem i komentarzem	id	praktyka_id, zmieniony_przez

Wszystkie tabele zawieraja kolumny techniczne created_at i updated_at (tam gdzie to uzasadnione) rejestrujace czas utworzenia i ostatniej modyfikacji rekordu. Tabela historia_statusu sluzy jako pelny audit log zmian stanu praktyki — kazda zmiana statusu jest rejestrowana automatycznie przez aplikacje Flask.